Exercise Sheet 10

Cascades and Temporal Networks

5942UE Network Science

26 March 2026

Prof. Dr. Michael Granitzer

10.A Cascades and Threshold Rules

Exercise 10.A.1: Threshold Cascade by Hand

Graph (edges A–B, B–C, A–D, C–E, D–F) with thresholds: $A(0.4), B(0.5), C(0.3), D(0.5), E(0.5), F(0.3)$.

Seed at $t = 0$: A, C. Synchronous update.

1. Compute the degree of each non-seed node.
2. Trace which nodes adopt at $t = 1, 2, 3$.
3. Does the cascade reach all nodes?

Solution:

1. Degrees: B=2, D=2, E=1, F=1.

2. $t = 1$:

- B: 2/2 = 1.0 ≥ 0.5 → **adopts**
- D: 1/2 = 0.5 ≥ 0.5 → **adopts**
- E: 1/1 = 1.0 ≥ 0.5 → **adopts**
- F: 0/1 = 0.0 < 0.3 → not yet

$t = 2$: F: 1/1 = 1.0 ≥ 0.3 → **adopts**.

3. Yes — full adoption by $t = 2$. F is last because it required D to adopt first.

Exercise 10.A.2: Threshold Heterogeneity

Same graph as 10.A.1. Two experiments:

- **A:** All thresholds raised to 0.6.
- **B:** Only B's threshold raised to 0.6 (others as in 10.A.1).

For seed A, C:

1. Does the cascade reach all nodes in (A)?
2. Does the cascade reach all nodes in (B)?
3. Why does a single threshold change "block" the cascade in (A)?

Solution:

1. Experiment A: D has 1/2 = 0.5 < 0.6 → **does not adopt**. F depends on D, also blocked. Cascade fails to reach D, F.

2. Experiment B: B still has 2/2 = 1.0 ≥ 0.6 → adopts. All other thresholds unchanged → full cascade reaches all nodes.

3. In (A), D requires both neighbours adopted. D's only neighbours are A (seed) and F. F won't adopt until D does — circular dependency. **Dense local clusters help complex cascades; isolated low-degree nodes with high thresholds block spread.**

Exercise 10.A.3: Cascade Simulation

Simulate threshold cascades on `nx.barabasi_albert_graph(100, 3)`.

1. At what seed size does the cascade reliably sweep most of the network?
2. Raise threshold to 0.5 — how does final adoption change?
3. Compare seeding **hubs** (top-k degree) vs **random**.

Solution:

```python
import networkx as nx
import random

def threshold_cascade(G, seeds, thresholds):
    adopted = set(seeds)
    changed = True
    while changed:
        changed = False
        for node in G.nodes():
            if node in adopted:
                continue
            nbrs = list(G.neighbors(node))
            if not nbrs:
                continue
            frac = sum(1 for n in nbrs if n in adopted) / len(nbrs)
            if frac >= thresholds[node]:
                adopted.add(node)
                changed = True
    return adopted
```

With $\theta = 0.3$, even 3–5 seeds can trigger a global cascade thanks to hubs. At $\theta = 0.5$ random cascades almost always fail — crossing 50% requires dense local reinforcement. **Hub seeding** is much more effective than random because hubs apply pressure to many neighbours at once.

Exercise 10.A.4: Bridges Differ for Simple vs. Complex Contagion

Dumbbell: two 5-cliques $1..5$ and $6..10$ joined by bridge 5–6. Node 1 adopts first.

1. **Simple contagion** ($\beta = 1$): does it spread to clique B?
2. **Complex contagion** ($\theta = 0.4$): does it cross the bridge?
3. What change would let the cascade cross?

Solution:

1. Yes — once node 5 is infected (quickly within clique A), the infection crosses to node 6 and rapidly fills clique B. Bridges are **superhighways** for simple contagion.

2. Within clique A nodes have degree 4; once 2 neighbours adopt (50% > 40%) adoption spreads. But node 6 has degree 4, with **only** its bridge neighbour (5) adopted: 1/4 = 25% < 40%. **Cascade does not cross.**

3. Add a second bridge (e.g., 4–7); or lower θ for bridge-adjacent nodes; or plant a seed in clique B. Bridges that help simple contagion *hinder* complex contagion by providing too little adoption pressure on the far side.

10.B Temporal Networks

Exercise 10.B.1: Order-Dependent Reachability

Three nodes A, B, C with two timed edges:

- A–B exists at $t = 1$ only
- B–C exists at $t = 2$ only

1. Can information travel $A \to C$? Trace the path.
2. Can it travel $C \to A$?
3. Reverse the timing (A–B at $t = 2$, B–C at $t = 1$). Does $A \to C$ still work?
4. What does this say about temporal vs. static reachability?

Solution:

1. Yes. $A \xrightarrow{t=1} B \xrightarrow{t=2} C$.

2. No. At $t = 1$ the B–C edge does not yet exist; at $t = 2$ the A–B edge no longer exists. No time-respecting path.

3. No — B–C at $t = 1$ comes too early (A has no link yet); A–B at $t = 2$ comes too late.

4. Temporal reachability is **asymmetric and order-dependent** even with undirected edges. The same nodes and edges can support completely different information flows depending on timing.

Exercise 10.B.2: Static Aggregation Hides Causal Order

A hospital contact network over 3 days:

Day	Edge
1	Nurse–Patient A
2	Nurse–Patient B
3	Doctor–Nurse

1. Can infection spread from Patient A to Doctor? Trace it.
2. Can it spread from Doctor to Patient A?
3. What might static contact tracing wrongly conclude?

Solution:

1. Yes. $\text{Patient A} \xrightarrow{d=1} \text{Nurse} \xrightarrow{d=3} \text{Doctor}.$

2. No. Doctor only contacts Nurse on day 3 — both patient contacts happened on days 1 and 2, before the doctor was exposed.

3. A static contact-tracing system sees Doctor–Nurse–PatientA–PatientB as one connected component and would suggest **bidirectional risk**. The real risk is unidirectional (patients → nurse → doctor). Wrong people could be quarantined while real exposure direction is missed.

Exercise 10.B.3: Temporal Reachability in Code

Implement BFS over a temporal edge list.

1. From node 0, which nodes are reachable and at what earliest time?
2. From node 3, is node 0 reachable?
3. Add edge $(3, 0, 5)$. How does reachability change?

Solution:

```python
from collections import deque

def temporal_reachable(edges, source, t_start=0):
    """BFS over (u, v, t) edges; an edge is traversable iff
       t >= current arrival time at u."""
    arrival = {source: t_start}
    queue = deque([(t_start, source)])
    while queue:
        t_curr, node = queue.popleft()
        for (u, v, t) in edges:
            if t < t_curr:
                continue
            nb = v if u == node else u if v == node else None
            if nb is not None and nb not in arrival:
                arrival[nb] = t
                queue.append((t, nb))
    return arrival

temporal_edges = [(0, 1, 1), (1, 2, 2), (2, 3, 3), (3, 0, 1), (0, 2, 4)]
```

Adding a late edge $(3, 0, 5)$ provides no new reachability if the nodes already met at an earlier time, but it would create new $3 \to 0$ reachability if no earlier time-respecting path existed. Reversing time would break the forward chain entirely.

Exercise 10.B.4: When Timing Kills Spreading

Star with hub H and leaves A, B, C, D. Disease starts at A at $t = 0$.

- **Scenario X:** all edges active at $t = 1$
- **Scenario Y:** H–A at $t = 1$, H–B at $t = 2$, H–C at $t = 3$, H–D at $t = 4$

1. Which nodes get infected in (X)?
2. In (Y)?
3. Now suppose the disease has a **1-step infectious window**. In (Y), does it spread beyond H?
4. What does this illustrate about epidemic containment?

Solution:

1. All nodes infected at $t = 1$ — simultaneous contact via H.

2. $A \to H$ at $t = 1$, $H \to B$ at $t = 2$, then C at $t = 3$, D at $t = 4$. Eventually all infected.

3. With a 1-step window, H is only infectious at $t = 1$. The H–B edge appears at $t = 2$, after H has stopped being infectious. B, C, D **are never infected.**

4. Even in a fully connected star, **temporal mis-alignment between contact timing and infectious window** can stop spread cold. This is the basis for **contact scheduling** (e.g., staggered shifts in hospitals) as an infection-control strategy.

Wrap-Up

- threshold cascades depend on dense local reinforcement, not just connectivity
- bridges are good for simple contagion but bad for complex cascades
- temporal networks make reachability asymmetric and order-dependent
- contact timing relative to infectious windows can decisively contain spread